

Multi-Protocol Traffic Generation and Analysis: A Containerized System for HTTP/2, QUIC, MQTT and Raw TCP/UDP

Amine Ahamri
Mobile & Internet Computing
Hochschule Rhein-Waal

Abstract—This report describes a containerized system that generates and controls network traffic across five protocols at once: HTTP/2, QUIC, MQTT and raw TCP/UDP. Ten Docker containers are coordinated by a central controller that reads YAML traffic profiles and steers four generators through a REST API. The system supports constant, periodic-burst, linearly ramped and Poisson-distributed sending patterns, an autonomous Adaptive Control loop that reacts to error rates and latency, and a stealth mode for the TCP/UDP generator that hides its statistical fingerprint. This report covers the architecture, the implementation decisions behind it, the configuration format, and the Wireshark-based traffic analysis.

Index Terms—HTTP/2, QUIC, MQTT, Docker, traffic generation, Wireshark, behavioral fingerprinting, adaptive control

I. INTRODUCTION

A normal office network never carries just one protocol. At any moment there is web traffic, IoT sensor data, video calls and background sync jobs all sharing the same wire. Studying how these protocols behave together, how they fail, and how easy they are to tell apart from the outside, needs a testbed that can actually produce that mix on demand and let you turn the dials while it runs.

This project builds exactly that testbed, based on the assignment in [1]. It generates traffic for five protocols at the same time: HTTP/2 [2], QUIC/HTTP-3 [3], [4], MQTT [5] with all three QoS levels, and raw TCP and UDP. Everything runs as ten Docker containers orchestrated with Docker Compose [6], started with one `docker-compose up`. A central controller loads a YAML profile and tells four traffic generators what to send and how fast; a web dashboard lets you change any of that live, without restarting anything.

Beyond the basic rate and payload-size knobs, the system implements four sending patterns (constant, periodic burst, linear ramp and Poisson-distributed random intervals), a stealth mode that makes the raw TCP/UDP generator statistically indistinguishable from normal traffic, and an autonomous Adaptive Control loop that scales a generator's rate up or down based on the error rate it is currently seeing, with no human in the loop.

The rest of this report is organized as follows. Section II describes the architecture. Section III walks through the implementation decisions that mattered, and why they were made that way. Section IV documents the configuration format. Sections V through VII present the Wireshark-based traffic analysis, the failure-visibility results and the single- versus multi-machine comparison. Section VIII reflects on what the data actually showed about protocol behavior, and Section IX documents how AI tools were used while building this.

II. SYSTEM ARCHITECTURE

The system has ten containers on one Docker bridge network called `mic-net`. Containers find each other by service name through Docker's internal DNS, so `gen-http2` can simply call `http://target-http2:8080` without anyone configuring an IP address anywhere.

The ten containers fall into four groups. The **controller** (FastAPI, port 8000) is the brain: it reads the YAML profile, drives the four generators through their REST APIs and runs the Adaptive Control loop. The four **generators** (`gen-http2`, `gen-quic`, `gen-mqtt`, `gen-tcpudp`) each keep their own state and expose the same four endpoints: `start`, `stop`, `patch the config`, `get the status`. They are built on `httpx` (HTTP/2), `aiouic` [7] (QUIC), `paho-mqtt` [8] (MQTT) and plain Python sockets (raw TCP/UDP). The three **target services** (`target-http2`, `target-quic`, `target-tcpudp`) plus the Mosquitto broker [9] simply receive traffic and, in two of the three cases, report what they received back to the metrics collector. `target-http2` runs on Hypercorn [10], the only ASGI server in the project that supports HTTP/2 natively. The **metrics collector** (Flask, port 9090) aggregates everything into one `/metrics` response, and the **dashboard** (a single static HTML file served over port 3000) polls the controller every two seconds and renders the live state.

Fig. 1 shows how a request actually travels through the system. The dashboard never talks to a generator directly; every change goes through the controller, which is what makes

it possible to log every configuration change with a timestamp, as the assignment requires.

One detail worth pointing out: `target-tcpudp` does not report to the metrics collector at all, unlike the other two targets. It only accepts connections and discards the data. This is deliberate, since the point of that container is to make a TCP RST visible in Wireshark when it goes down, and adding a reporting path would only get in the way of that.

III. IMPLEMENTATION DECISIONS AND TRADE-OFFS

A. Connection reuse vs. a fresh connection per packet

The HTTP/2 generator opens one `httpx` client with HTTP/2 enabled and keeps it open for the whole run, firing several requests at once over that single connection to actually use multiplexing rather than just claiming to. The QUIC generator does the same thing for the same reason: a QUIC handshake costs at least one extra round trip, and if the generator reconnected for every request that handshake cost would swamp the latency numbers and make QUIC look artificially slow next to HTTP/2 and MQTT, both of which also keep a connection open.

The TCP/UDP generator does the opposite on purpose. For every single TCP packet it opens a new connection, sends, and closes it again. This was not an oversight. A fresh connect/close cycle produces a real SYN, SYN-ACK and FIN sequence in the capture, which is exactly what is needed later for the Failure Visibility analysis and for telling raw TCP apart from the application-level protocols in a protocol hierarchy view.

B. Mode and pattern as two separate knobs

The TCP/UDP generator splits "what does a packet look like" from "how is it paced over time". `mode` controls size and base interval: normal mode uses a fixed 512 bytes every 100 ms, stealth mode draws a random size between 64 and 1400 bytes with Poisson-distributed timing. `pattern` independently controls the sending cadence on top of that: constant, periodic burst, or an extra random gap. Keeping these two orthogonal made it possible to test, for example, stealth-mode sizing combined with a burst cadence, which would not have been possible if mode and pattern were the same setting.

C. The Poisson pattern on every protocol, not just one

The assignment only asks for a random, Poisson-distributed pattern somewhere in the system. This implementation puts it on all four generators using `random.expovariate(1/interval)` (or NumPy's exponential distribution for the TCP/UDP generator), which produces exponentially distributed gaps between sends with the same average rate as the constant pattern. The reason to extend it everywhere was to be able to show one single Wireshark capture where HTTP/2, QUIC and MQTT all use Poisson timing at once, alongside TCP/UDP stealth mode, rather than only ever demonstrating it on one protocol.

TABLE I
PER-PROTOCOL CONFIGURATION FIELDS

Protocol	Fields
http2	rate, payload_size, method_get_pct, concurrent_streams, get_paths, post_paths, pattern, burst_rate, burst_duration, burst_interval, fault_rate, extra_latency_ms
quic	rate, payload_size, stream_count, use_0rtt, pattern, fault_rate, extra_latency_ms
mqtt	rate, payload_size, qos, topic_count, qos_distribution, pattern, fault_rate, extra_latency_ms
tcp/udp	tcp_rate, udp_rate, packet_size, mode, mean_interval, min_size, max_size, tcp_ratio, pattern, burst_size, burst_interval, fault_rate, extra_latency_ms

D. Warmup and cooldown double as the ramp pattern

Rather than building a separate "ramp" mode per protocol, the warmup and cooldown periods are handled once, centrally, in the controller. A function called `_ramp_rates()` steps every generator's rate field linearly from zero up to its configured target over the warmup window, and back down to zero over the cooldown window, sending the new value through the normal PATCH endpoint at each step. This single mechanism satisfies both requirements at once: the warmup/cooldown behavior the assignment asks for, and the "ramping" sending pattern, since the gradual change shows up as a slope in a Wireshark I/O graph either way.

E. Adaptive Control

The controller can run an autonomous control loop, either because a YAML phase turns it on or because it was switched on manually. Every few seconds it checks each generator's error rate, and where reported its latency, against configurable thresholds, and scales the rate up or down by a multiplier bounded on both ends. This goes past the baseline requirements, but it ties directly into the resilience patterns covered in the course: the system reacts to an injected failure on its own, without anyone touching a slider.

IV. CONFIGURATION FORMAT

The controller reads multi-phase YAML profiles from the `config/` folder. Every profile has a `global` block with the total duration, warmup and cooldown in seconds, and a list of phases. Each phase has a name, a duration, an optional `adaptive_control` block, and a `protocols` block with one entry per protocol that should be active during that phase. Three profiles ship with the project: `balanced.yaml` (constant baseline, then stealth plus random pattern, then adaptive control), `http2_heavy.yaml` (HTTP/2 under load with a burst phase), and `mqtt_heavy.yaml` (a QoS 0 vs. QoS 2 vs. mixed comparison).

Field names in the YAML have to match the generator's Pydantic model exactly. Anything that does not match is silently ignored rather than rejected, since the models accept extra keys for forward compatibility but the generators only apply the ones they actually recognize. One concrete example of this showing up in the shipped profiles: a few phases list an

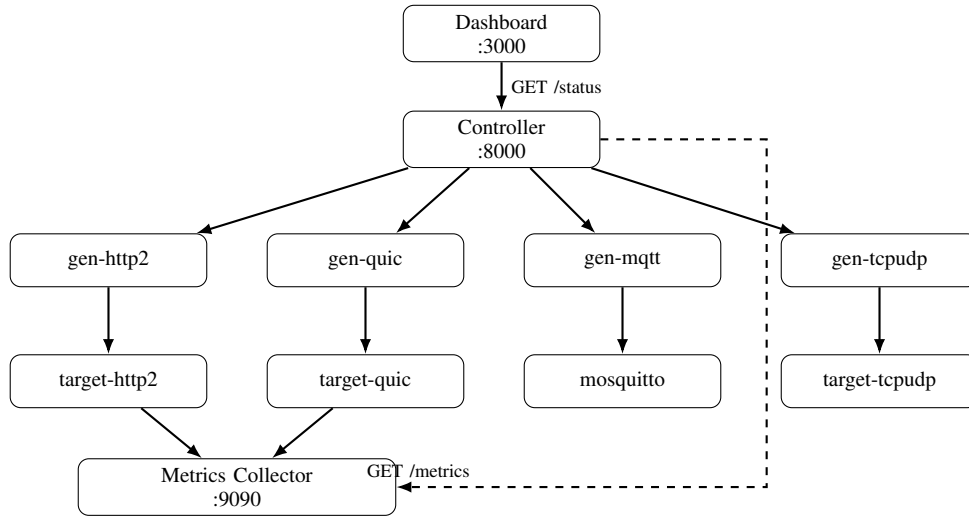


Fig. 1. Component diagram of the single-machine deployment. Solid arrows are REST calls or raw protocol traffic; the dashed arrow is the controller reading aggregated metrics for status reporting and Adaptive Control.

explicit topics array for readability, but the MQTT generator only ever looks at topic_count. The named list currently has no effect at runtime. This is a known gap rather than something hidden, and it is worth fixing if there is time left before submission.

Configuration is not locked to the YAML once the system is running. POST /config/load switches the active profile and applies phase one immediately if the system is already started. PATCH /generator/{name} forwards any subset of the fields in Table I to one generator. The dashboard wraps every one of these fields in a slider, toggle or text input that calls the same two endpoints, debounced so that dragging a slider does not flood the controller with requests.

V. TRAFFIC ANALYSIS RESULTS

This section presents the five required Wireshark captures. Each subsection below has the exact setup already filled in, since that part does not depend on the actual capture, only on what to insert once the capture exists.

A. Protocol Distribution

Capture at least 60 seconds with the balanced profile running all four generators. Export Statistics → Protocol Hierarchy as a screenshot.

[INSERT] Screenshot of the Protocol Hierarchy and the percentage breakdown of TCP vs. UDP, and within TCP the HTTP/2 (port 8080), MQTT (port 1883) and raw TCP shares, plus the UDP-only QUIC share (port 4433).

B. Temporal Analysis

Capture 120 seconds with the http2_heavy profile, which includes a periodic-burst phase (burst rate 400 for 5 seconds every 30 seconds). Export Statistics → I/O Graph with one line per protocol filter.

[INSERT] I/O graph screenshot with the burst peaks marked, plus a second capture of a warmup period and a Random/Poisson phase for comparison against the constant and burst shapes.

C. Behavioral Fingerprinting

Capture two 30-second windows of the TCP/UDP generator alone: one in mode=normal, one in mode=stealth. Optionally extend this with the balanced_with_stealth phase, which runs stealth mode together with the random pattern on HTTP/2, QUIC and MQTT at the same time.

[INSERT] Packet-length histograms and inter-arrival-time distributions for both modes, with an explicit statement of which characteristics are distinguishable (e.g. the fixed 100 ms spacing in normal mode) and which overlap with real client traffic.

D. Failure Visibility

Capture across a docker stop target-http2 event during normal operation. Filter on tcp.flags.reset==1 for the TCP RST, or look for repeated SYNns to port 1883 if stopping mosquitto instead.

[INSERT] Measured time from the stop command to the first visible failure indicator, and which protocol-specific signal appeared. See Section VI for the deeper discussion of this capture together with the Adaptive Control reaction.

E. Multi-System Deployment

Capture on the physical network interface of either lab machine (not docker0/br-*) while docker-compose.targets.yml runs on Machine B and docker-compose.generators.yml runs on Machine A with TARGET_B_IP pointing at Machine B.

[INSERT] Capture screenshot plus measured latency per protocol, compared against an equivalent single-machine run. See Section VII for the full comparison.

VI. FAILURE ANALYSIS RESULTS

[INSERT] Consolidated discussion of the Failure Visibility capture from Section V-D. Cover: the per-protocol failure signal observed (TCP RST, MQTT reconnect attempts, QUIC connection-close frames), how long it took to become visible, and what the unaffected protocols looked like during the outage. If Adaptive Control was enabled during the same run, add the controller's logged decisions (error rate, multiplier, action) and how quickly the affected generator's rate was scaled down and back up once the target recovered.

VII. SINGLE-MACHINE VS. MULTI-MACHINE COMPARISON

[INSERT] Quantitative table: latency_ms per protocol on a single machine (loopback) vs. on two machines (real Ethernet hop). Add a short note on what only becomes visible in the multi-machine capture: real host IP addresses instead of Docker's 172.x.x.x range, no shared Docker bridge, and genuine Ethernet framing under load.

VIII. REFLECTION ON PROTOCOL BEHAVIOR UNDER DIFFERENT TRAFFIC PATTERNS

This section should pull together what Sections V through VII actually showed, not repeat them. Questions worth answering once the data is there:

- Which protocol was easiest to fingerprint in normal, non-stealth conditions, and why?
- Did the random pattern change the I/O graph shape equally for every protocol, or did some (HTTP/2 firing several streams per cycle, for instance) keep some residual periodicity?
- Did the measured packet-to-message ratio for MQTT match the expected 1x for QoS 0 and 4x for QoS 2?
- How did Adaptive Control's reaction time compare to the raw, protocol-level failure detection time (TCP RST in under a second vs. the configured check interval)?
- What would change about distinguishability if TLS were used everywhere, including MQTT and raw TCP?

[INSERT] Two to four paragraphs answering the points above with concrete numbers from your own captures.

IX. AI USAGE DOCUMENTATION

I used Anthropic's Claude throughout this project, working directly in the repository through an agentic coding session with file and shell access.

For the code, I used it to scaffold the first version of the `docker-compose.yml` layout, the controller's FastAPI skeleton and the four generators' REST contract (start, stop, patch config, get status), and then iterated on that by hand and through more prompts. The Adaptive Control loop, the warmup/cooldown ramp, the random pattern on all four generators and the per-protocol fault injection were built the same way, one function at a time, with a Python syntax check after every change. The OpenAPI/Swagger documentation was generated from the Pydantic models and FastAPI decorators

and kept in sync with a small regeneration script run after every API change.

For the documentation, I had it review the README and the docs against the actual code and fix what no longer matched, for example an old claim that the TCP/UDP generator used Scapy when the real implementation just uses plain sockets.

This report itself was drafted the same way: the architecture, implementation and configuration sections came from the verified source code, while the traffic analysis, failure analysis and multi-machine sections were left as placeholders on purpose, since those need real Wireshark captures that only I can produce on lab hardware.

I reviewed every change before accepting it and ran the syntax checks myself. I also went through the entire codebase module by module in a separate walkthrough session to make sure I could explain every part of it on my own, not just recognize that it works.

[INSERT] Replace this paragraph with your own honest account before submission, including anything I missed above.

REFERENCES

- [1] Hochschule Rhein-Waal, "Project: Multi-protocol traffic generation and analysis," Mobile & Internet Computing course assignment, Summer Term 2026.
- [2] M. Belshe, R. Peon, and M. Thomson, "Hypertext transfer protocol version 2 (http/2)," RFC 7540, IETF, 2015.
- [3] J. Iyengar and M. Thomson, "Quic: A udp-based multiplexed and secure transport," RFC 9000, IETF, 2021.
- [4] M. Bishop, "Http/3," RFC 9114, IETF, 2022.
- [5] OASIS, "Mqtt version 5.0," OASIS Standard, 2019.
- [6] Docker Inc., "Docker compose documentation," <https://docs.docker.com/compose/>.
- [7] aioquic contributors, "aioquic: A quic and http/3 implementation in python," <https://github.com/aiohttp/aioquic>.
- [8] Eclipse Foundation, "paho-mqtt: Eclipse paho mqtt python client," <https://github.com/eclipse/paho.mqtt.python>.
- [9] —, "Eclipse mosquitto: An open source mqtt broker," <https://mosquitto.org/>.
- [10] P. Jones, "Hypercorn: An asgi/wsgi server," <https://github.com/pgjones/hypercorn>.